

CSCI433/933 Machine Learning Algorithms and Applications

Assignment 2: Domain Adaptation with Small Language Models

Group Project Specification

Instructor: Professor Philip O. Ogunbona

Due date: Midnight, Friday May 29

Assignment Theme

This assignment requires each group to design, implement, and evaluate a Shakespeare-aware language system using a pretrained Small Language Model (SLM), retrieval-augmented generation, and structured evaluation. The emphasis is on practical domain adaptation under realistic computational and organisational constraints.

Introduction and Motivation

Recent advances in natural language processing have been strongly influenced by large-scale foundation models. However, many real-world organisations do not require, cannot afford, or cannot govern very large externally hosted models for every downstream task. There is therefore growing interest in **Small Language Models (SLMs)** that are computationally efficient, easier to adapt, and suitable for controlled deployment within corporate, government, educational, or research environments.

SLMs are particularly attractive where an organisation requires a purpose-built system for a bounded task, such as internal document question answering, technical support, policy search, customer assistance, training support, or knowledge-grounded dialogue. In such contexts, the objective is not to build the largest possible model, but to design a reliable, auditable, and task-focused language system that can operate under constraints involving cost, privacy, latency, maintainability, and domain specificity.

This assignment explores that paradigm through a controlled domain adaptation task. Each group will adapt a pretrained language model to support interaction with selected Shakespearean plays. The task is deliberately chosen because Shakespearean text differs significantly from contemporary English in vocabulary, syntax, idiom, and cultural context. At the same time, the assignment does *not* assume that all students have previously studied Shakespeare. The system should therefore be designed to help a beginner understand and interact with the plays.

Pedagogical Focus

This assignment evaluates your ability to design, adapt, and evaluate a language model-based system under constraints. It is not an exercise in training a large model from scratch or competing on GPU resources. Nor is it an exercise to maximise model size or computational scale.

Problem Statement

Each group must design and implement a **Shakespeare-aware language system** that enables a user with little or no prior exposure to Shakespeare to ask questions, retrieve relevant evidence, and receive useful explanations or short stylised responses.

The system must demonstrate the following capabilities:

1. **Understanding:** provide clear explanations of characters, events, relationships, and themes.
2. **Question answering:** answer user questions about the selected plays accurately and concisely.
3. **Grounded retrieval:** retrieve and display relevant excerpts from the source text.
4. **Stylised generation:** generate short responses in a Shakespearean style, while clearly distinguishing creative generation from evidence-based explanation.

Technical Requirements and Constraints

Model Constraints

Students must use a **pretrained SLM** or an approved hosted language model. The purpose of the assignment is to adapt and evaluate an existing model, not to pretrain a model from scratch.

Important Constraint

Training a language model from scratch is **not permitted**. Full-scale pretraining, distributed training, custom CUDA engineering, or large-scale hyperparameter search are outside the scope of this assignment.

Acceptable adaptation strategies include:

- prompt engineering;
- retrieval-augmented generation (RAG);
- lightweight fine-tuning, such as LoRA or QLoRA, where feasible;
- hybrid approaches combining retrieval and controlled generation.

If using a hosted model, groups must justify how their system design aligns with the principles of small, deployable, or resource-constrained models.

Mandatory Architecture

A **Retrieval-Augmented Generation (RAG)** pipeline is mandatory. At minimum, the system must include:

1. document preparation and chunking;
2. embedding-based indexing;
3. retrieval of relevant passages for a user query;
4. generation conditioned on the retrieved context;
5. display of retrieved evidence alongside, or immediately below, the generated answer.

Retrieved passages must be demonstrably used in generating responses. Systems that do not meaningfully integrate retrieved context into generation will be penalised.

Compute Constraints

The system must be designed to run on either:

- a standard personal computer; or
- free-tier or modest cloud infrastructure.

Heavy GPU dependence is not expected. Marks will not be awarded for using unnecessarily large models or complex infrastructure. Marks will be awarded for appropriate design, clear justification, correctness, evaluation, and reflective analysis.

Dataset Constraints

Each group must use the following **three compulsory plays**:

1. *Hamlet*
2. *Macbeth*
3. *Romeo and Juliet*

Groups are not required to use the entire text verbatim and may subset or further prepare the data as appropriate, provided the approach is justified.

The processed dataset must represent each play using a structured format such as:

Play → Act → Scene → Speaker → Utterance

Groups may use supplementary material such as scene summaries, character summaries, glossaries, or explanatory notes. However, all supplementary sources must be clearly documented in the report. The report must distinguish between primary Shakespearean text and supporting explanatory material.

Accessibility Requirement

The system should be useful to a user who has not previously studied Shakespeare. Therefore, explanations should avoid assuming prior literary knowledge.

System Requirements

The implemented system must support the following four interaction types.

Concept Explanation

The system should answer questions such as:

- Who is Hamlet?
- What is the role of Lady Macbeth?
- What is the conflict between the Montagues and the Capulets?

Contextual Question Answering

The system should answer questions requiring understanding of events and motivations, such as:

- Why does Macbeth kill Duncan?
- Why does Hamlet delay taking revenge?
- Why is Juliet conflicted after meeting Romeo?

Evidence Retrieval

For each substantive answer, the system should retrieve relevant source passages. The retrieved evidence should include available metadata such as play, act, scene, speaker, and excerpt.

Stylised Response Generation

The system should be able to generate a short response in a Shakespearean style. Stylised responses must be limited to **150 words**. The system should not present creative stylised output as factual evidence. Stylised responses must remain interpretable and should not sacrifice clarity for stylistic imitation.

Evaluation Requirements

Each group must conduct a structured evaluation using **10–15 evaluation questions**.

The evaluation set must contain:

- **5 instructor-provided questions**; and
- **5–10 group-designed questions** created by the student group.

The instructor-provided questions will establish a common basis for comparison across groups. The group-designed questions will allow each group to test additional aspects of its system, including robustness, style control, retrieval quality, and usefulness for beginners.

Each group must implement:

- a baseline system (e.g., prompt-only or minimal retrieval);
- an improved system (RAG-based).

The report must include a comparison between the baseline and the improved system.

Evaluation Criteria

Each group must evaluate the system using the following criteria:

1. **Correctness:** Is the answer factually accurate with respect to the selected plays?
2. **Grounding:** Is the answer supported by retrieved evidence?
3. **Retrieval relevance:** Are the retrieved passages appropriate for the question?
4. **Usefulness:** Would the answer help a beginner understand the relevant character, event, or theme?
5. **Style quality:** For stylised generation, does the response reasonably reflect Shakespearean tone without becoming incomprehensible?

Scoring

Groups should define a simple and consistent scoring scale (e.g., 1–5 or low/medium/high) for each evaluation criterion and apply it consistently across all evaluation questions.

The report must include both successful examples and failure cases. Failure analysis is an important part of the assignment and should discuss hallucination, irrelevant retrieval, incomplete answers, misleading style generation, or poor handling of ambiguous questions.

Marks are awarded based on the quality of reasoning, justification, and analysis, not merely on whether outputs appear plausible.

Group Structure and Timeline

This is a group assignment for groups of **four students**. A recommended role distribution is shown in Table 1.

Role	Indicative Responsibility
Data Engineering Lead	Corpus acquisition, cleaning, segmentation, metadata structure, dataset documentation.
Model Development Lead	Baseline model, RAG pipeline, prompt design, optional lightweight fine-tuning.
Evaluation Lead	Evaluation questions, scoring rubric, error analysis, comparison of baseline and improved system.
Integration and Reporting Lead	System interface, repository organisation, report integration, demonstration preparation.

Table 1: Suggested group role distribution. Groups may organise themselves differently, but all members must contribute substantially.

Five-Week Timeline

Week	Milestone
Week 1	Dataset preparation, task definition, repository setup, initial baseline plan.
Week 2	Baseline system implemented using prompting or a simple retrieval approach.
Week 3	Working RAG pipeline with indexed Shakespeare corpus and retrieved evidence.
Week 4	Evaluation, refinement, error analysis, and optional extension work.
Week 5	Final system, report, demonstration, code repository, and GenAI usage log submitted.

Table 2: Recommended five-week project timeline.

Use of Generative AI Tools

Students are permitted to use generative AI tools such as ChatGPT, Copilot, or similar systems to support brainstorming, coding, debugging, explanation, and report drafting. However, use of such tools must be transparent and reflective.

The GenAI usage log must include:

- prompts used;
- concise summaries of useful responses;
- how the responses influenced the project;
- what was verified, modified, rejected, or improved by the group.

Direct copying of AI-generated material without understanding, verification, or acknowledgement will be penalised.

Assessment Criteria

Component	Marks
Problem formulation and task understanding	10
Dataset preparation and documentation	15
Model adaptation and RAG methodology	20
System implementation and usability	15
Evaluation, comparison, and error analysis	20
Technical report quality	10
Responsible and transparent GenAI usage	10
Total	100

Table 3: Assessment weighting.

System Execution and Assessment

To ensure fairness and feasibility, assessment will be based on a combination of demonstration, documentation, and code review.

Execution Requirements

Each group must ensure that their system is **demonstrable and reproducible** under the following conditions:

- The system must run on a standard laptop or equivalent environment.
- The system must not require specialised hardware beyond what is stated in the report.
- All dependencies must be clearly listed.

Caching embeddings

Groups may cache embeddings, retrieval indices, and intermediate results. Recomputation of embeddings or indexing during assessment is not required.

Submission Requirements

Each group must submit the following components as part of the assignment:

1. **Working system:** A runnable prototype (e.g., Python script or command-line interface) demonstrating the required interaction types. The system must clearly demonstrate the use of retrieved evidence in generating responses.
2. **Source code repository:** Clearly organised code with:
 - setup instructions;
 - execution steps;
 - description of system components;
3. **Technical report:** A LaTeX-formatted report following the provided template. The report must clearly describe the problem, dataset, methodology, implementation, evaluation, and limitations.
4. **Evaluation appendix:** A complete record of evaluation questions, retrieved evidence, system responses, scoring, and error analysis.
5. **Demonstration:** A short video (5–10 minutes) or live demonstration showing:
 - system overview;
 - example queries;
 - retrieved evidence;
 - generated responses;
6. **GenAI usage log:** A transparent record of generative AI usage, including prompts, summaries, and verification steps.

Technical Report Requirement

The technical report is a mandatory and assessable component of the submission.

The main report must not exceed 10 pages (excluding references, appendices, and GenAI usage log), consistent with IEEE-style conference submissions.

What Will Be Assessed

Assessment will be based on:

- correctness and quality of system outputs;
- quality of retrieval and grounding;
- clarity and justification of design decisions;
- quality of evaluation and error analysis;
- code organisation and reproducibility;
- alignment between reported results and demonstrated behaviour.

Optional Live Demonstration

The instructor may request a short live demonstration for verification purposes. Students must be able to:

- run the system;
- answer questions about design choices;
- demonstrate key functionality.

Full re-execution of training or indexing pipelines during assessment is **not required**.

Important Notes

- The assignment is about domain adaptation, retrieval, evaluation, and system integration.
- Do not train a model from scratch.
- Do not spend excessive time on GPU configuration or infrastructure engineering.
- Do not assume that the user of the system has prior Shakespeare knowledge.
- Students are not required to build a full user interface, web application, or production system. A simple command-line or Python script interaction is sufficient.
- Always distinguish evidence-based answers from creative stylised generation.
- Marks are awarded for clarity, correctness, justification, evaluation, and reflective analysis rather than unnecessary complexity.

Final Reminder

A successful submission does not need to be the most complex system. It must be a well-designed, well-evaluated, and clearly explained system that demonstrates how a small or constrained language model can be adapted for a specific downstream task.

Appendix

Practical Implementation Guide

This appendix provides practical guidance on how to approach the assignment. It is intended to reduce unnecessary uncertainty about tools, workflow, and implementation choices. It does not prescribe a single correct solution. Groups are still expected to make, justify, and evaluate their own design decisions.

Recommended Development Workflow

A sensible workflow for this assignment is as follows:

1. Inspect the three Shakespeare plays provided for the assignment.
2. Inspect the provided structured dataset and decide how it will be represented for retrieval.
3. Prepare the selected records, fields, and metadata for indexing.
4. Split the text into meaningful retrieval units, such as scenes, speaker turns, or short passages.
5. Generate embeddings for each retrieval unit.
6. Build a retrieval index using a vector search method.
7. Connect the retriever to a SLM using a Retrieval-Augmented Generation pipeline.
8. Test the system using the instructor-provided and group-designed questions.
9. Analyse failures and refine the system.
10. Prepare the final report, demonstration, source code, and GenAI usage log.

Important clarification on preprocessing A structured version of the Shakespeare dataset is provided to ensure consistency across groups and to reduce low-level data cleaning effort.

However, groups are still expected to make and justify design decisions regarding how the data is used within the system. This may include:

- redefining or refining chunk boundaries;
- selecting or modifying which fields are used for retrieval;
- augmenting the data with summaries or additional metadata;
- filtering or restructuring records for improved retrieval quality.

Therefore, “preprocessing” in this assignment refers to thoughtful preparation and representation of the dataset for retrieval and generation, not raw text cleaning.

Data Preparation

Although a structured dataset is provided, groups must decide how to prepare, select, chunk, and represent the data for retrieval and generation. At minimum, each utterance or passage should preserve the following metadata:

- play title;
- act number;
- scene number;
- speaker name, where available;
- original utterance or passage;
- optional modern-English summary or explanation.

A recommended JSON-style record is shown below:

```
{
  "play": "Macbeth",
  "act": "Act 1",
  "scene": "Scene 3",
  "speaker": "MACBETH",
  "utterance": "So foul and fair a day I have not seen.",
  "modern_summary": "Macbeth comments on the strange and contradictory nature of the day."
}
```

Groups may use Python tools such as regular expressions, **pandas**, **json**, and text-processing libraries to clean and structure the data. If external summaries or supporting materials are used, their sources must be documented in the report.

Chunking Strategy

The retrieval system depends strongly on how the text is divided into chunks. Groups should avoid arbitrary splitting that destroys context. Suitable chunking strategies include:

- one chunk per scene;
- one chunk per speaker turn;
- fixed-size overlapping text chunks;
- semantically meaningful passages grouped by event or topic.

Each chunk should retain metadata so that retrieved evidence can be traced back to the play, act, scene, and speaker.

Choice of Model

Students are not expected to train a large model from scratch. The assignment should be completed using a pretrained SLM or an approved hosted model.

Acceptable options include:

- a hosted SLM accessed through an API;
- a locally run small model using a framework such as HuggingFace Transformers;

- a quantised small model suitable for laptop or free-tier cloud execution;
- a RAG system where the main adaptation is achieved through retrieval and prompt design;
- optional lightweight fine-tuning, such as LoRA or QLoRA, for groups seeking an extension.

Full model pretraining is not permitted. Marks are awarded for sound design, justification, implementation, and evaluation, not for using the largest possible model.

Embedding Models and Retrieval Tools

Groups may use lightweight embedding models such as:

- `sentence-transformers/all-MiniLM-L6-v2`;
- `BAAI/bge-small-en-v1.5`;
- other instructor-approved embedding models.

Possible retrieval/indexing tools include:

- FAISS;
- Chroma;
- LlamaIndex;
- LangChain;
- a simple NumPy-based cosine similarity implementation.

A sophisticated retrieval framework is not required. A simple, well-understood implementation with clear evaluation is preferable to an unnecessarily complex pipeline.

Prompt Design

The generation component should be prompted to use retrieved context rather than relying only on its general prior knowledge. A useful prompt should instruct the model to:

- answer using the retrieved passages;
- indicate uncertainty when the evidence is insufficient;
- avoid inventing unsupported details;
- provide beginner-friendly explanations;
- produce stylised Shakespearean output only when requested.

Groups should include representative prompts in the report or appendix and explain how the prompts evolved during development.

Evaluation Procedure

Evaluation must include both instructor-provided questions and group-designed questions. For each question, groups should record:

- the question;
- the retrieved passage or passages;
- the generated answer;
- correctness score;
- grounding score;
- style score, where applicable;
- usefulness score for a beginner user;
- comments on strengths, weaknesses, and failure cases.

A suggested evaluation table is:

question, expected_focus, retrieved_passage, response,
correctness, grounding, style, usefulness, comments

Suggested Project Folder Structure

The following folder structure is recommended:

```
assignment2/  
  data/  
    raw/  
    processed/  
  src/  
    preprocess.py  
    build_index.py  
    rag_chatbot.py  
    evaluate.py  
  prompts/  
    system_prompt.txt  
    evaluation_prompts.txt  
  results/  
    evaluation_results.csv  
  report/  
    assignment2_report.tex  
  README.md
```

Groups may modify this structure, but the submitted repository should be organised and easy to run.

Starter Code Specification

Starter code may be provided to reduce setup friction. The starter code is not intended to solve the assignment. It provides a scaffold that groups must complete, modify, and justify.

Purpose of the Starter Code

The starter code should help students:

- organise the project directory;
- load and inspect the Shakespeare corpus;
- define a standard data schema;
- create text chunks for retrieval;
- build a basic embedding index;
- run a simple retrieval query;
- connect retrieved passages to a language model;
- record evaluation results.

Students are expected to extend the starter code and make their own design choices.

Example Data Schema

A processed record may be represented as follows:

```
record = {
    "play": "Macbeth",
    "act": "Act 1",
    "scene": "Scene 3",
    "speaker": "MACBETH",
    "text": "So foul and fair a day I have not seen.",
    "source_id": "macbeth_1_3_macbeth_001"
}
```

Example Preprocessing Skeleton

```
def load_raw_text(file_path):
    """
    Load a raw Shakespeare play from a text file.
    """
    with open(file_path, "r", encoding="utf-8") as f:
        return f.read()

def parse_play(raw_text, play_title):
    """
    Convert raw play text into structured records.

    Students should implement parsing logic to identify acts,
    scenes, speakers, and utterances.
    """
    records = []

    # TODO: implement parsing logic
```

```
    return records

def save_processed_records(records, output_path):
    """
    Save processed records as JSON.
    """
    import json

    with open(output_path, "w", encoding="utf-8") as f:
        json.dump(records, f, indent=2, ensure_ascii=False)
```

Example Retrieval Skeleton

```
def create_chunks(records):
    """
    Convert structured records into retrieval chunks.

    Each chunk should preserve useful metadata such as play,
    act, scene, and speaker.
    """
    chunks = []

    # TODO: define chunking strategy

    return chunks

def embed_chunks(chunks, embedding_model):
    """
    Generate vector embeddings for each chunk.
    """
    texts = [chunk["text"] for chunk in chunks]
    embeddings = embedding_model.encode(texts)

    return embeddings

def retrieve(query, chunks, embeddings, embedding_model, top_k=3):
    """
    Retrieve the top-k most relevant chunks for a query.

    Students may implement cosine similarity, FAISS, Chroma,
    or another approved retrieval method.
    """
    # TODO: implement retrieval

    retrieved_chunks = []

    return retrieved_chunks
```

Example RAG Skeleton

```
def build_prompt(query, retrieved_chunks):
    """
    Build a prompt that conditions the language model on
    retrieved Shakespeare passages.
    """
    context = "\n\n".join(
        [chunk["text"] for chunk in retrieved_chunks]
    )

    prompt = f"""
    You are a Shakespeare-aware assistant.

    Use the retrieved context below to answer the user's question.
    If the context is insufficient, say so clearly.
    Explain the answer in a way that helps a beginner.

    Retrieved context:
    {context}

    User question:
    {query}

    Answer:
    """
    return prompt

def generate_answer(prompt, language_model):
    """
    Generate an answer using the selected language model.
    """
    # TODO: connect to local model or approved API

    answer = ""

    return answer
```

Example Evaluation Skeleton

```
def evaluate_question(question, expected_focus, system):
    """
    Run one evaluation question through the system and
    return the retrieved evidence and generated answer.
    """
    retrieved_chunks = system.retrieve(question)
    response = system.generate(question, retrieved_chunks)

    result = {
        "question": question,
        "expected_focus": expected_focus,
```

```
        "retrieved_passage": retrieved_chunks,  
        "response": response,  
        "correctness": None,  
        "grounding": None,  
        "style": None,  
        "usefulness": None,  
        "comments": ""  
    }  
  
    return result
```

What Students Must Add

Students must not merely submit the starter code unchanged. They are expected to add:

- a working preprocessing pipeline;
- a justified chunking strategy;
- a working retrieval component;
- a language model interface;
- a complete RAG pipeline;
- evaluation results;
- error analysis;
- documentation explaining how to run the system.

Minimum Viable Implementation

A minimum viable implementation should be able to:

1. ingest the three required plays as a dataset and make them available to the retrieval component;
2. prepare and represent them as structured records for retrieval;
3. retrieve relevant passages for a user query;
4. generate a beginner-friendly answer using retrieved context;
5. display the retrieved evidence;
6. evaluate the system on the required question set.

High Distinction-Level Extensions

Possible extensions include:

- comparison of two embedding models;
- comparison of two chunking strategies;
- retrieval re-ranking;

- lightweight LoRA or QLoRA fine-tuning;
- hallucination analysis;
- improved user interface;
- structured character or scene knowledge graph;
- systematic prompt ablation study.

These extensions are optional. A simple system that is well designed, correctly implemented, carefully evaluated, and clearly reported can still receive a high mark.

Grading Rubric

This rubric describes the expected standard of performance for each grade band. Marks are awarded based on the quality of reasoning, clarity of explanation, soundness of design decisions, and depth of evaluation.

Grade Bands

Grade	Descriptor
HD (85–100)	Demonstrates deep understanding and strong technical execution. The system is well-designed, clearly implemented, and thoroughly evaluated. Retrieval and generation are meaningfully integrated, and evaluation includes insightful analysis and well-explained failure cases. Design decisions are well justified, and the report is clear, concise, and professional.
D (75–84)	Demonstrates solid understanding and competent implementation. The system works well and includes appropriate evaluation. Some insightful analysis is present, though depth or clarity may be uneven. Design decisions are generally justified, and the report is well structured.
C (65–74)	Demonstrates adequate understanding with a functional system. Evaluation is present but largely descriptive rather than analytical. Retrieval and generation may be loosely integrated or inconsistently effective. Justification of design decisions is limited or partially developed.
P (50–64)	Demonstrates limited understanding. The system may be incomplete, poorly integrated, or weakly evaluated. Evaluation lacks depth, and analysis is minimal. Important aspects of the assignment may be missing or unclear.
F (<50)	Does not meet the minimum requirements of the assignment. The system is incomplete or incorrect, with little or no meaningful evaluation. Key components such as retrieval, grounding, or comparison are missing.

Criteria-Based Guidance

The following guidance describes how the grade bands apply to each assessed component.

C.2.1 Problem Formulation (10 marks)

- **HD:** Clear, precise, and well-motivated problem definition aligned with domain adaptation and SLM constraints.
- **D:** Clear problem definition with good alignment to task requirements.
- **C:** Basic problem description with limited depth or clarity.

- **P:** Vague or incomplete problem formulation.
- **F:** Problem not clearly defined or misunderstood.

C.2.2 Dataset Preparation (15 marks)

- **HD:** Well-structured dataset with clear schema, justified preprocessing, and thoughtful chunking strategy.
- **D:** Good dataset preparation with appropriate structure and preprocessing.
- **C:** Dataset prepared but with limited justification or suboptimal structure.
- **P:** Minimal or poorly explained preprocessing.
- **F:** Dataset handling missing, incorrect, or undocumented.

C.2.3 Model Adaptation and RAG Methodology (20 marks)

- **HD:** Strong, well-justified RAG design with clear integration of retrieval and generation.
- **D:** Effective RAG implementation with reasonable justification.
- **C:** Basic RAG implementation with limited integration or explanation.
- **P:** Weak or incomplete implementation of RAG concepts.
- **F:** RAG not implemented or fundamentally incorrect.

C.2.4 System Implementation (15 marks)

- **HD:** Clean, well-organised, reproducible system with clear structure and usability.
- **D:** Functional system with good organisation.
- **C:** Working system with some issues in clarity or structure.
- **P:** Partially working or difficult-to-run system.
- **F:** System not working or not submitted.

C.2.5 Evaluation and Analysis (20 marks)

- **HD:** Comprehensive evaluation with clear metrics, strong comparison, and insightful analysis of successes and failures.
- **D:** Good evaluation with some meaningful analysis.
- **C:** Evaluation present but mostly descriptive.
- **P:** Minimal evaluation with little analysis.
- **F:** Evaluation missing or incorrect.

C.2.6 Technical Report (10 marks)

- **HD:** Clear, well-structured, concise, and professionally presented report.
- **D:** Well-organised report with minor clarity issues.
- **C:** Adequate report with some structural or clarity problems.
- **P:** Poorly organised or unclear report.
- **F:** Report missing or unreadable.

C.2.7 Responsible Use of Generative AI (10 marks)

- **HD:** Transparent, reflective, and critical use of GenAI with clear documentation.
- **D:** Clear documentation of GenAI use with some reflection.
- **C:** Basic documentation with limited reflection.
- **P:** Minimal or unclear documentation.
- **F:** No documentation or inappropriate use.

Important Notes

- Marks are awarded for depth of understanding, justification, and evaluation quality, not system complexity.
- A simple system that is well designed and well analysed can achieve a high grade.
- A complex system with weak evaluation or poor explanation will not achieve a high grade.